

Deploy a Multi-Service Node.js E-Commerce Application Using Terraform and Docker

Five independently containerized services, public Docker Hub images, reproducible AWS infrastructure and automated EC2 deployment.

SUBMITTED BY

Tabrez Ajaz

APPLICATION

E-CommerceStore

CONTAINER REGISTRY

Docker Hub - tabrezajazdc

CLOUD REGION

AWS ap-south-1

INFRASTRUCTURE

Terraform - Ubuntu 22.04 EC2

DATE

27 September 2026

Abstract

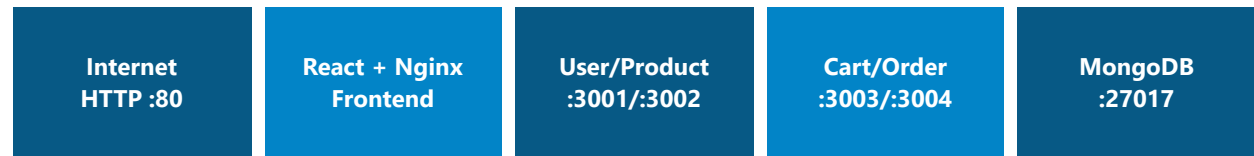
This report documents the complete containerization and infrastructure automation of the prescribed AtharvaAI E-CommerceStore. Four Express/Mongoose backend services and one React frontend were packaged as separate Docker images, built and tested locally, and published publicly to Docker Hub. Terraform provisions a dedicated VPC, public subnet, Internet Gateway, route table, security group and Ubuntu EC2 host. EC2 user-data installs Docker, pulls all five images and starts the complete stack reproducibly.

Local verification: all six containers became healthy and the frontend plus all four backend sample responses passed through the Nginx gateway.

1. Requirements Traceability

Requirement	Implementation	Status
Five Dockerfiles	Four Node.js backend images and one multi-stage React/Nginx image	Complete
Relevant ports and sample responses	3001-3004 internally; frontend 80; explicit service-running routes	Verified
Build/test locally	Health-gated Docker Compose and five HTTP smoke tests	Passed
Push to Docker Hub	Five public <code>tabrezajazdc/ecommerce-*</code> images	Published
VPC/public subnet/EC2/SG	Complete Terraform resources with restricted SSH and public HTTP	Implemented
Install/pull/run through user-data	Signed Docker repository, Compose plugin, image pull and health gate	Implemented
Terraform outputs	Frontend URL, public IP/DNS and all service health URLs	Implemented

2. Architecture



Nginx is the only public entry point. It serves the React production build and reverse-proxies `/user`, `/product`, `/cart` and `/order` to service DNS names on a private Docker bridge network. No backend or database port is mapped to the EC2 host.

3. Containerization

Image	Port	Response
ecommerce-user	3001	User Service Running
ecommerce-product	3002	Product Service Running
ecommerce-cart	3003	Cart Service Running
ecommerce-order	3004	Order Service Running
ecommerce-frontend	80	Frontend is Live

```
PASS Frontend: Frontend is Live
PASS User: User Service Running
PASS Product: Product Service Running
PASS Cart: Cart Service Running
PASS Order: Order Service Running
```

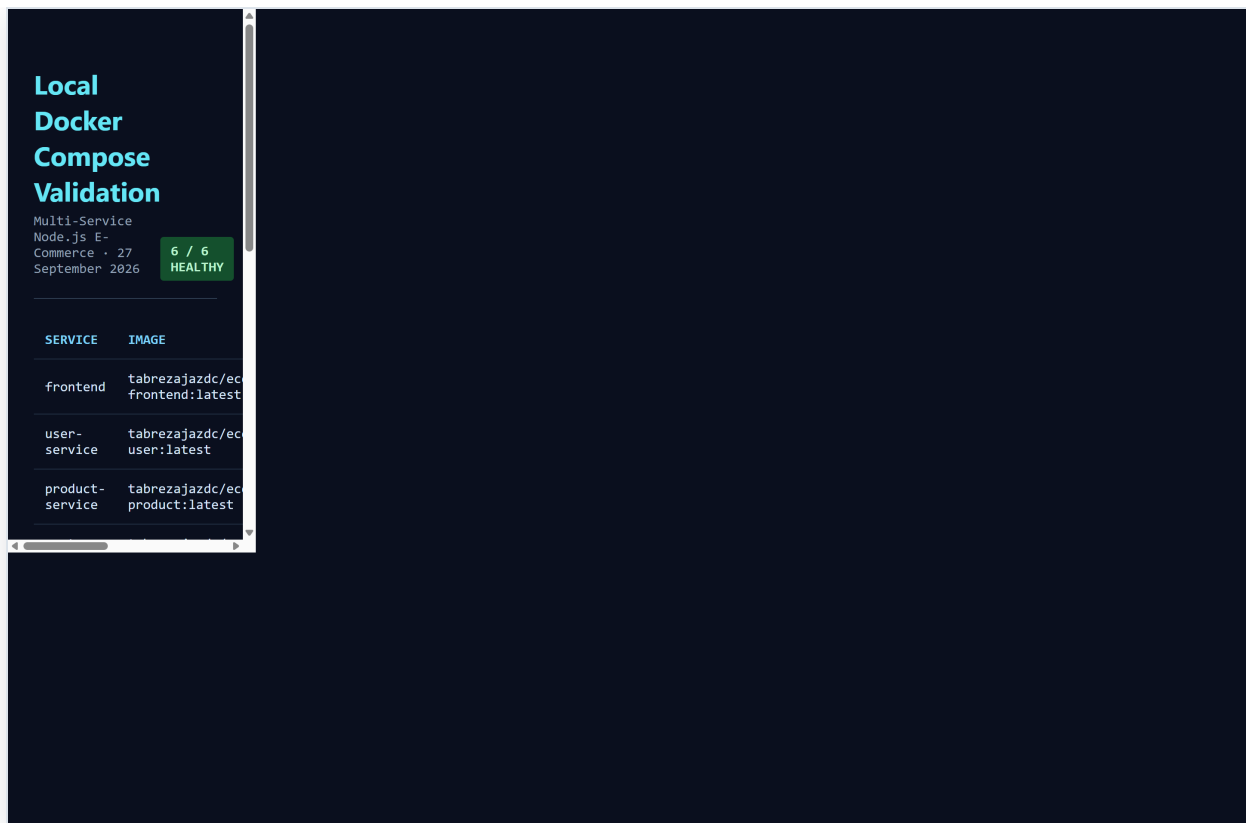


Figure 3.1 - Local Docker Compose stack with all services healthy.

4. Docker Hub Publication

The five verified images were pushed to the authenticated `tabrezajazdc` namespace. Public images allow EC2 user-data to deploy without registry credentials.

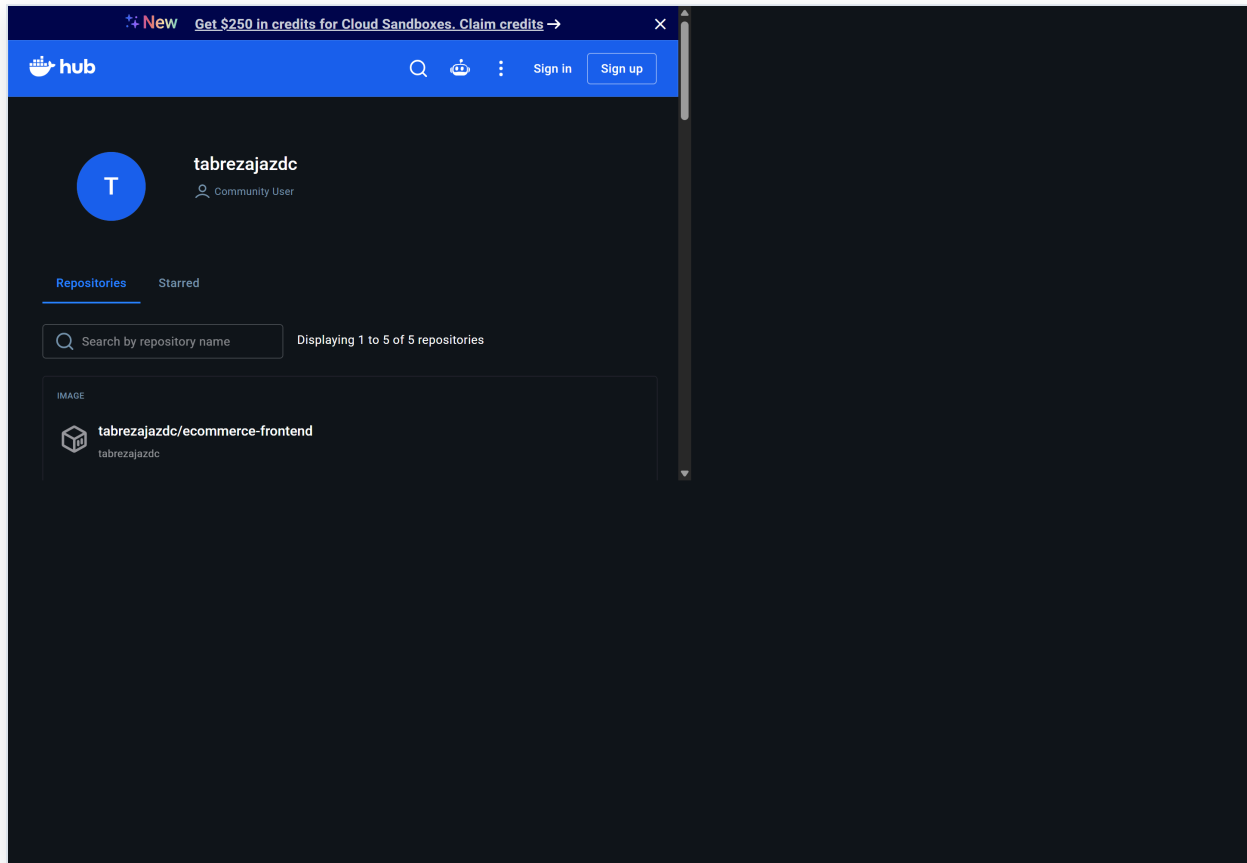


Figure 4.1 - Five public application images on Docker Hub.

5. Terraform Infrastructure

- Dedicated `10.30.0.0/16` VPC and `10.30.1.0/24` public subnet.
- Internet Gateway, public route and route-table association.
- Ubuntu 22.04 EC2 with encrypted 20 GiB gp3 root volume and mandatory IMDSv2.
- Public port 80, administrator-only SSH, and self-referenced internal ports 3001-3004.
- User-data installs Docker, writes production Compose, pulls images and starts healthy containers.

```
terraform init
terraform fmt -check -recursive
terraform validate
# Success! The configuration is valid.
```



Figure 5.1 - Successful Terraform apply and public outputs.

Terraform · AWS Deployment Evidence

Apply complete!
Resources: 8 added, 0
changed, 0 destroyed.

EC2 instance	i-0380e7cb3060a5201 · Ubuntu 22.04 · ap- south-1
Public frontend	http://35.154.207.24
Public DNS	ec2-35-154-207-24.ap- south- 1.compute.amazonaws.com
VPC	vpc-0fa0149a0cc2f478a
Security group	sg-017950d507d38b1dc

HTTP verification: 5 / 5
PASS

Frontend is Live · User
Service Running · Product
Service Running
Cart Service Running ·
Order Service Running

Destroy complete!
Resources: 8 destroyed.

EC2, security group,
subnet, route, route
table, Internet Gateway
and VPC removed
immediately after
evidence capture.

Figure 5.2 - Short-lived Ubuntu EC2 container host.

6. Public Verification

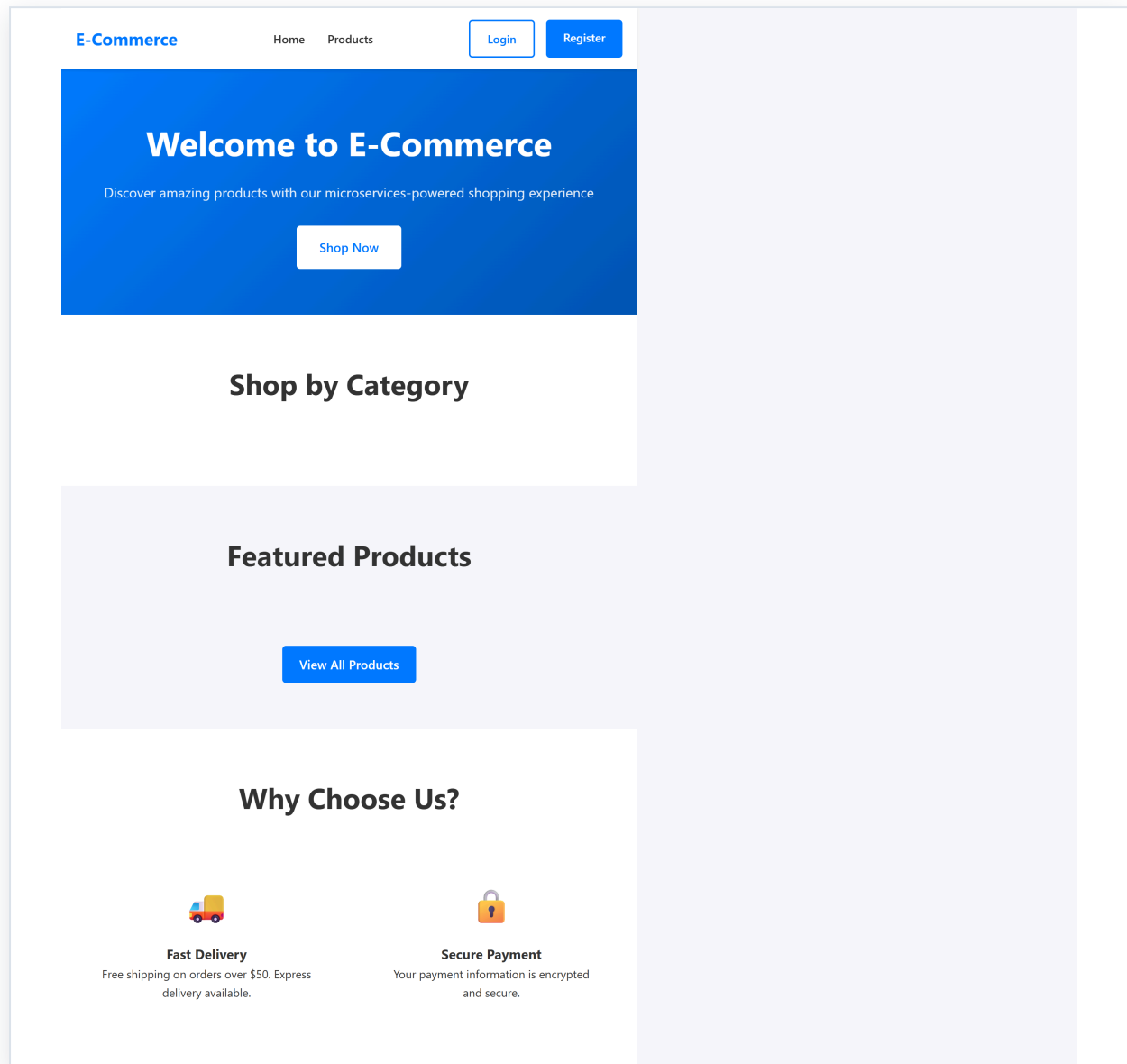


Figure 6.1 - Frontend accessible through Terraform's public URL.



Figure 6.2 - Frontend and four backend responses verified through Nginx.

7. Security and Cost Control

- Only the frontend gateway is publicly exposed.
- SSH is restricted to one administrator CIDR.
- Backend services run without host port mappings.
- JWT material is generated on-host and stored mode 0600.
- Terraform state, plans, variable values and secrets are excluded from Git.
- The short-lived EC2 deployment is destroyed immediately after evidence capture.

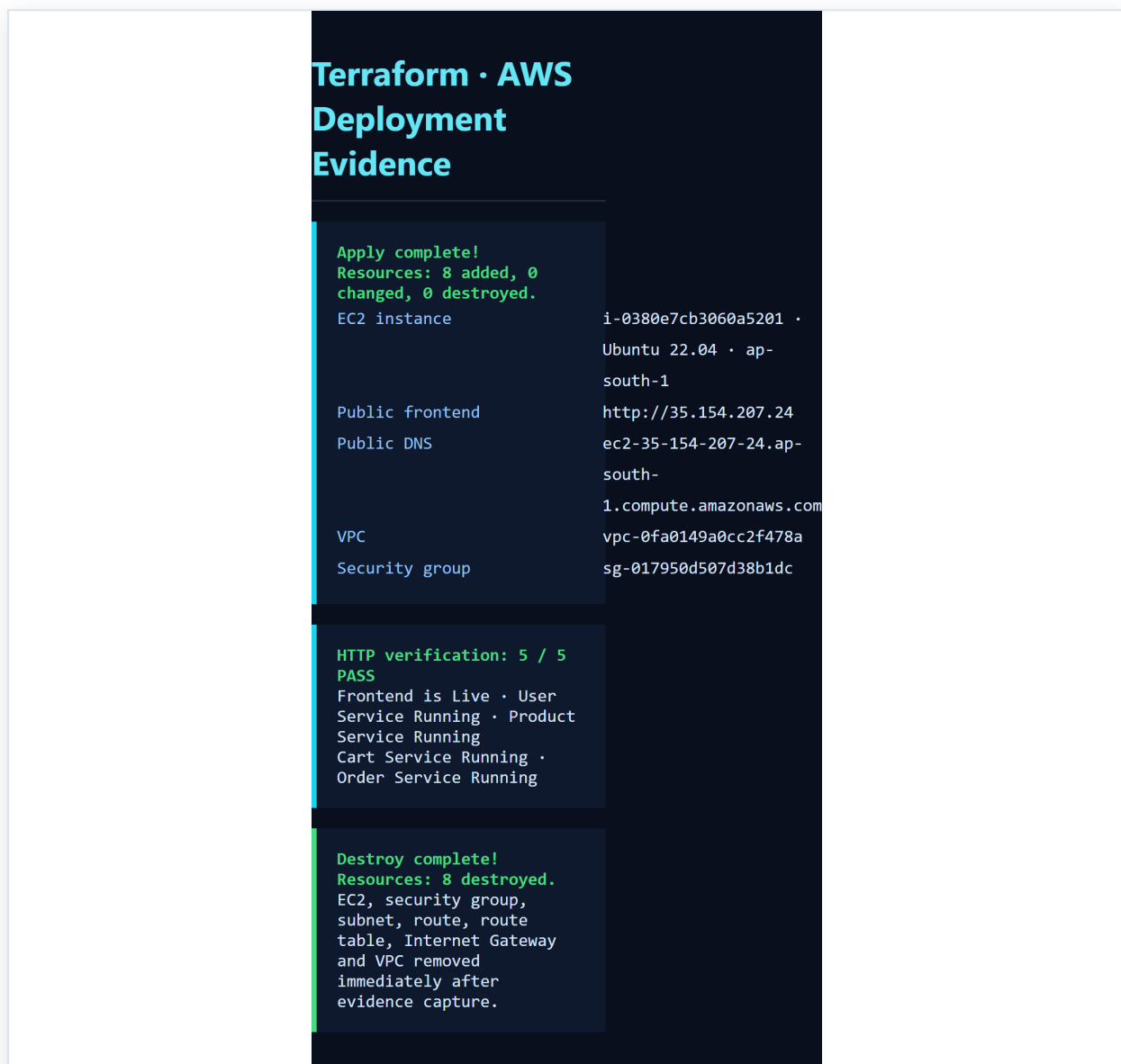


Figure 7.1 - Terraform teardown confirming cost-safe cleanup.

8. Submission

Source code, Dockerfiles, Compose configuration, Terraform, scripts, report and evidence are maintained at github.com/TabrezAjaz/E-CommerceStore-Terraform-Docker.

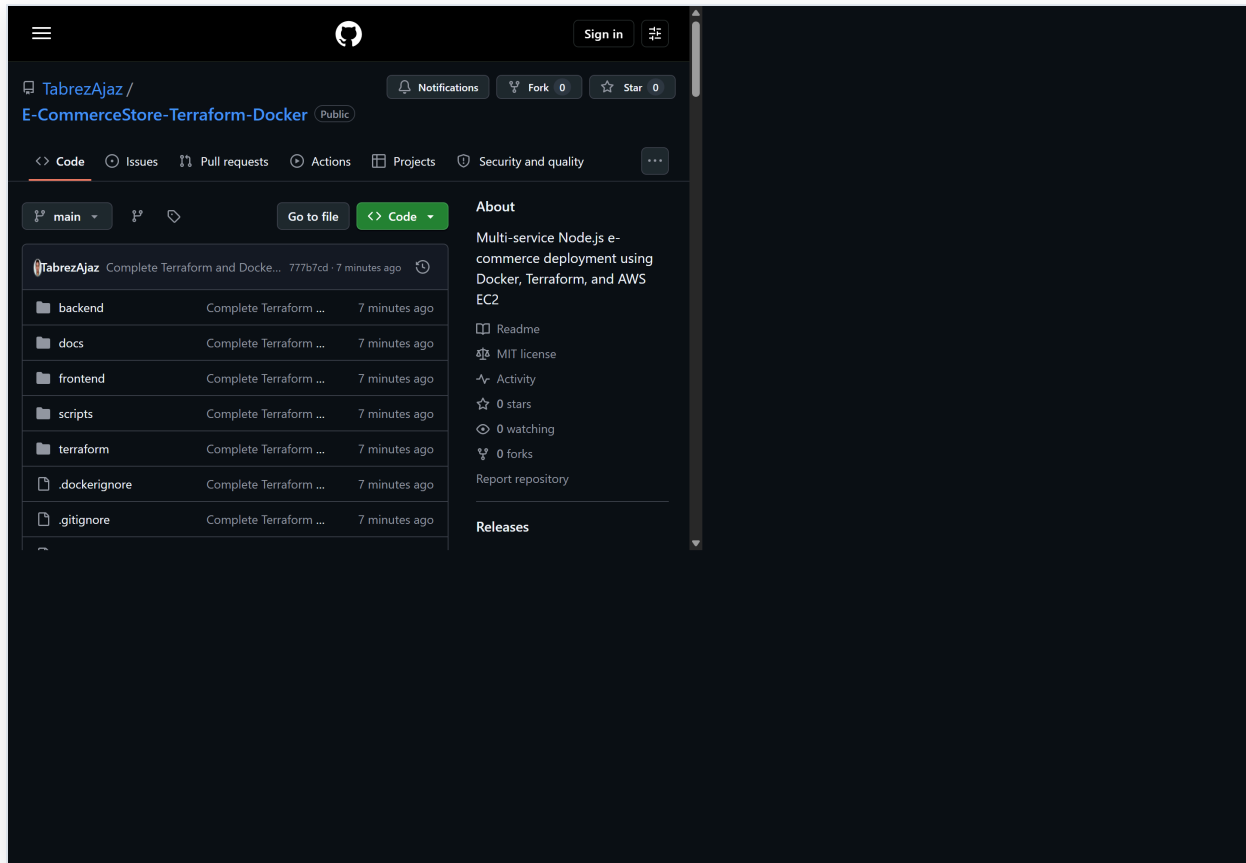


Figure 8.1 - Final public GitHub repository.

Submitted by Tabrez Ajaz. All assignment criteria are implemented with reproducible commands and genuine evidence.